

Generación de consultas SQL a partir de lenguaje natural aplicado al ERP Fail Fast

Beatriz Natalia Serrano, Martín Santos y Luis Gabriel Moreno

Resumen—La generación automática de consultas SQL a partir de lenguaje natural (Text-to-SQL) representa un desafío fundamental en el procesamiento del lenguaje natural, especialmente en entornos empresariales complejos como los sistemas ERP. Este trabajo presenta una arquitectura híbrida multi-agente que integra descomposición de consultas, recuperación contextual del esquema y generación controlada de SQL mediante modelos de lenguaje de gran escala (LLMs). La arquitectura propuesta distribuye la tarea entre agentes especializados: un Selector que identifica sub-esquemas relevantes, un Decomposer que divide consultas complejas en subtarefas manejables, y un Refiner que ejecuta y corrige iterativamente las consultas generadas. La evaluación se realizó sobre el sistema ERP Fail Fast, utilizando métricas estándar como Execution Accuracy (EX), Time-to-Answer (TTA) y Component Matching. Los resultados demuestran mejoras significativas ***** en precisión (incremento del 15 % en EX) y robustez ***** comparado con enfoques monolíticos, validando la hipótesis de que la colaboración multi-agente y la recuperación contextual mejoran sustancialmente el rendimiento en esquemas complejos. La arquitectura desarrollada democratiza el acceso a la información empresarial, permitiendo consultas en lenguaje natural sin requerir conocimientos técnicos especializados.

IMPACT STATEMENT

Index Terms—Text-to-SQL, sistemas multi-agente, LLM, ERP, procesamiento de lenguaje natural, RAG, descomposición de consultas, inteligencia artificial empresarial

I. INTRODUCCIÓN

LA generación de SQL, también conocida como Text-to-SQL, es una tarea fundamental del procesamiento del lenguaje natural cuyo propósito es convertir preguntas formuladas en lenguaje natural (NL) en consultas SQL ejecutables sobre bases de datos relacionales. Su objetivo central es democratizar el acceso a la información, permitiendo que usuarios sin conocimientos técnicos interactúen con los datos mediante expresiones propias de su lenguaje cotidiano [1].

Este proceso exige la comprensión profunda de tres elementos: la intención del usuario, la estructura del esquema de base de datos y la construcción correcta de la sintaxis SQL. En el contexto organizacional, y especialmente en sistemas de planificación de recursos empresariales (ERP), el acceso oportuno a la información es crítico para la toma de decisiones.

Beatriz Natalia Serrano y Martín Santos son estudiantes de la Maestría en Inteligencia Artificial, Facultad de Ingeniería, Pontificia Universidad Javeriana, Bogotá, Colombia.

Luis Gabriel Moreno (Ph.D.) es Jefe de Sección de Ingeniería de Software, Facultad de Ingeniería, Pontificia Universidad Javeriana, Bogotá, Colombia, y director de tesis.

Este trabajo fue desarrollado como parte del programa de Maestría en Inteligencia Artificial de la Pontificia Universidad Javeriana (enero de 2026).

Tradicionalmente, dicho acceso se limita a reportes o tableros preconfigurados, que ofrecen una interacción unidireccional con la base de datos. Sin embargo, no existe un mecanismo nativo que permita a los usuarios solicitar información directamente en lenguaje natural, lo que restringe la flexibilidad y agilidad en el análisis.

La incorporación de capacidades Text-to-SQL plantea desafíos relevantes, entre ellos: interpretar con precisión la intención del usuario, resolver referencias implícitas (por ejemplo, reconocer que “ventas del mes pasado” corresponde al esquema “sales” y sus atributos temporales), controlar el acceso seguro a los datos, y manejar ambigüedades inherentes al lenguaje natural. Además, los desafíos propios de un ambiente multi-tenant agregan complejidad adicional, donde se accede a información de múltiples empresas que en muchos casos se trata de información confidencial, requiriendo mecanismos robustos de aislamiento de datos, control de acceso granular y auditoría de consultas para garantizar que cada organización solo pueda acceder a sus propios datos. Como se detallará en la sección correspondiente, estos retos convierten al Text-to-SQL en un campo de investigación complejo y estratégico para entornos empresariales contemporáneos.

I-A. Contexto y Motivación

La interacción entre los usuarios y los sistemas de planificación de recursos empresariales (ERP) ha estado históricamente limitada por la necesidad de comprender estructuras tabulares complejas y dominar lenguajes de consulta como SQL. Aunque estos sistemas gestionan con precisión las transacciones y el control operativo, su acceso cognitivo continúa siendo restringido para quienes no tienen formación técnica. Tras más de treinta años de experiencia en el desarrollo e implementación de ERP, Fail Fast ha evidenciado una brecha persistente: mientras las organizaciones requieren interpretar su información en lenguaje natural para tomar decisiones, los sistemas siguen exigiendo consultas formales y conocimiento especializado.

Con la llegada de los modelos de lenguaje de gran escala (LLM), esta situación se transforma. Investigaciones recientes demuestran que los LLM pueden traducir instrucciones en lenguaje natural a consultas SQL con altos niveles de precisión [2], [3], abriendo la posibilidad de que los usuarios accedan directamente a la información empresarial mediante preguntas expresadas en su propio lenguaje.

No obstante, la literatura reciente evidencia limitaciones persistentes en enfoques basados en LLM para Text-to-SQL: el desempeño tiende a degradarse cuando los esquemas de datos son extensos o “huge”, debido a la interferencia de

información irrelevante y a la mayor dificultad de schema linking [5], [17]. Asimismo, existen fallas cuando la consulta requiere conocimiento implícito no descrito explícitamente en la pregunta o el esquema (p. ej., definiciones de métricas, reglas de negocio o fórmulas), lo que ha motivado marcos de knowledge-intensive Text-to-SQL y el uso de conocimiento externo [20], [16]. Finalmente, cuando las preguntas demandan razonamiento multietapa, se observan mejoras al emplear descomposición del problema y estrategias explícitas de razonamiento (p. ej., chain-of-thought y flujos multiagente), frente a enfoques directos [3], [24], [17].

La motivación de este trabajo, por tanto, se fundamenta en aprovechar la experiencia acumulada en sistemas ERP y las capacidades emergentes de la inteligencia artificial para permitir que los usuarios formulen consultas en lenguaje natural y obtengan respuestas precisas basadas en los datos de su organización.

I-B. Planteamiento del Problema

I-B1. Justificación: El interés por mejorar la generación de SQL a partir de lenguaje natural ha crecido de manera sostenida, especialmente desde que los modelos de lenguaje de gran escala demostraron capacidad para traducir instrucciones cotidianas en consultas formales. Sin embargo, en entornos reales como los sistemas ERP, la interacción con la información sigue siendo limitada para quienes no dominan estructuras tabulares o lenguajes como SQL. A pesar de que estos sistemas cumplen un papel central en la trazabilidad y el control de los procesos empresariales, todavía existe una brecha entre su complejidad interna y la forma en que los usuarios formulan sus preguntas.

Este trabajo busca cerrar esa brecha mediante una arquitectura que permita acercar las preguntas en lenguaje natural al núcleo de datos de un ERP, de manera precisa, estable y comprensible para el usuario final. Esta motivación se potencia gracias a las características del ERP Fail Fast, desarrollado durante el último año sobre un stack moderno —PostgreSQL, Python y React— que facilita la integración nativa de modelos de lenguaje y agentes inteligentes.

A diferencia de sistemas heredados, la base de datos de Fail Fast fue diseñada desde cero con una estructura coherente, documentada y con nomenclaturas alineadas al significado funcional de cada tabla. Esta decisión de ingeniería no es menor: los estudios recientes muestran que la claridad semántica en los esquemas y la consistencia en los nombres de tablas y columnas favorecen el schema linking y reducen errores de interpretación por parte de los LLM [5], [6].

I-B2. Objetivos: **Objetivo General:** Diseñar y evaluar una arquitectura híbrida multi-agente para la generación automática de consultas SQL a partir de lenguaje natural en sistemas ERP, que mejore significativamente la precisión y robustez comparado con enfoques monolíticos.

Objetivos Específicos:

1. Identificar los desafíos del proceso Text-to-SQL en el ERP Fail Fast, considerando su base de datos documentada, la semántica de sus tablas y columnas, y las particularidades del dominio empresarial.
2. Diseñar una arquitectura híbrida que integre la descomposición de consultas, la recuperación contextual del

esquema y la generación controlada de SQL mediante modelos de lenguaje, orquestadas a través de agentes de inteligencia artificial.

3. *****. Analizar el comportamiento de metodologías de última generación al aplicarlas sobre un sistema ERP real como Fail Fast, para comparar su rendimiento frente a benchmarks tradicionales como Spider o BIRD.
4. *****. Evaluar el desempeño de la arquitectura propuesta utilizando métricas consolidadas en la literatura —Execution Accuracy (EX), Schema Linking Accuracy (SLA), Context Retention Rate (CRR), Task Success Rate (TSR) y Time-to-Answer (TTA).
5. *****. Caracterizar los tipos de preguntas empresariales formuladas por los usuarios, con el fin de orientar futuras extensiones hacia interacciones multimodales y mecanismos que permitan democratizar el acceso seguro a la información.

I-B3. Hipótesis: ***** La hipótesis central de este trabajo es que una arquitectura híbrida basada en descomposición de consultas, recuperación contextual del esquema y generación controlada de SQL, orquestada mediante agentes de inteligencia artificial, mejorará significativamente la precisión y la robustez del proceso Text-to-SQL en un ERP real como Fail Fast, superando el desempeño de enfoques monolíticos basados únicamente en modelos de lenguaje.

Se espera que esta arquitectura:

- Incremente la Execution Accuracy (EX) al reducir errores de razonamiento y ambigüedad en consultas complejas
- Mejore la Schema Linking Accuracy (SLA) gracias a la utilización de contexto estructurado y a la semántica clara del modelo de datos de Fail Fast
- Preserve mejor el contexto conversacional (CRR) mediante agentes especializados y flujos de verificación
- Aumente la utilidad práctica del sistema, reflejada en la Task Success Rate (TSR)
- Reduzca tiempos de respuesta (TTA) debido a una orquestación que filtra y estructura adecuadamente la información que recibe el modelo

Con ello, se plantea que una arquitectura modular y multi-agente no solo es más adecuada para esquemas complejos como los de un ERP, sino que también constituye un camino efectivo para democratizar el acceso seguro a la información empresarial con interacción en lenguaje natural.

II. MARCO TEÓRICO Y ESTADO DEL ARTE

Al cierre de 2024 y durante 2025, distintos estudios de revisión [7], [8], [9] han evidenciado el avance sostenido de la generación automática de SQL mediante grandes modelos de lenguaje. Estas investigaciones consolidan el campo como un eje de convergencia entre el procesamiento del lenguaje natural, el razonamiento estructurado y la interacción con bases de datos.

En particular, Shi et al. [7] proponen una taxonomía integral de los enfoques LLM-based para Text-to-SQL, clasificándolos por tipo de prompting, nivel de supervisión, y estrategias de autocorrección. Por su parte, Liu et al. [8] resumen más de 250

artículos y definen el ciclo completo “modelo-datos-evaluación-errores”, enfatizando la necesidad de estrategias de validación semántica y debugging continuo en entornos reales, como los sistemas ERP.

Estos trabajos recientes refuerzan que el propósito de Text-to-SQL ha evolucionado desde la simple traducción NL→SQL hacia interfaces cognitivas de consulta, donde los LLM funcionan como agentes semánticos capaces de razonar, corregirse y aprender con retroalimentación de la base de datos [9].

II-A. Primeras Aproximaciones

Los primeros avances en Text-to-SQL se desarrollaron a partir de sistemas basados en reglas y plantillas, como NaLIR [10] y SQLizer [11]. Estos enfoques resultaban adecuados únicamente para escenarios simples, ya que ofrecían interpretabilidad y control, pero su capacidad de generalización era limitada y requerían un alto esfuerzo manual para construir y mantener reglas.

Posteriormente, con el auge del aprendizaje profundo, surgieron los modelos neuronales secuencia a secuencia (Seq2Seq), que introdujeron la posibilidad de aprender relaciones entre preguntas en lenguaje natural y consultas SQL [12], [1]. Estos modelos mostraron un avance importante frente a los métodos anteriores, al permitir la extracción automática de características y el modelado de mapeos complejos. Sin embargo, presentaban problemas de escalabilidad, dependencia de grandes volúmenes de datos etiquetados y tendencia al sobreajuste.

Una etapa siguiente estuvo marcada por el uso de modelos de lenguaje pre entrenados (PLMs), como BERT y T5, que permitieron transferir conocimiento lingüístico adquirido en grandes corpus hacia la tarea de generación de SQL [2], [13]. Estos modelos mejoraron la comprensión semántica y ofrecieron un marco más escalable, aunque aún requerían ajustes finos específicos y su rendimiento dependía de la calidad de los datos del dominio.

En la actualidad, los grandes modelos de lenguaje (LLMs) han alcanzado un papel protagonista en el campo. Su capacidad para procesar y generar texto similar al humano los hace altamente efectivos frente a la variabilidad lingüística, demostrando una notable capacidad de generalización entre dominios [9], [14]. Estos avances posicionan a los LLMs como la tendencia de próxima generación en interfaces de bases de datos.

II-B. Metodologías Basadas en LLMs para la Generación de SQL

A diferencia de las aproximaciones tempranas basadas en reglas, plantillas o modelos de lenguaje preentrenados (PLMs), esta sección se concentra en las metodologías recientes que emplean grandes modelos de lenguaje (LLMs) como eje principal para la generación de SQL. Estos enfoques representan el estado del arte actual y se caracterizan por el uso de prompting especializado, la descomposición modular de tareas, técnicas eficientes de ajuste fino y ciclos de retroalimentación iterativa que fortalecen la precisión y la capacidad de generalización entre dominios [5], [16], [15].

II-B1. Decomposition for Enhancing Attention (DEA-SQL): Xie et al. [15] introducen DEA-SQL, un enfoque de tipo workflow que descompone la tarea Text-to-SQL en pasos atómicos para focalizar la atención del LLM y mejorar su capacidad de resolución. El pipeline se organiza en cinco módulos: (1) Information Determination, que filtra ruido y selecciona atributos relevantes; (2) Classification & Hint, que identifica el tipo de problema y ajusta prompts específicos; (3) SQL Generation, apoyado en few-shot prompting y plantillas; (4) Self-Correction, orientado a errores frecuentes como joins y agregaciones; y (5) Active Learning, que integra ejemplos fallidos para expandir el alcance del modelo.

Este diseño busca superar las limitaciones del single-step prompting, donde los prompts largos difunden la atención y afectan la precisión. En sus experimentos, DEA-SQL mejora entre 2 y 3 puntos porcentuales respecto a los baselines en Spider Dev, Spider-Realistic y BIRD Dev, y alcanza resultados state of the art en Spider Test [15].

II-B2. DIN-SQL (Decomposed In-Context Learning with Self-Correction): DIN-SQL [3] propone un enfoque basado completamente en in-context learning (ICL) que divide la tarea Text-to-SQL en subtarear guiadas por prompts especializados y añade un módulo de autocorrección para reparar errores comunes sin necesidad de ajuste fino. A partir del análisis de 500 ejemplos de Spider, los autores identifican seis categorías de fallos frecuentes: errores de schema linking, joins incompletos o incorrectos, uso inadecuado de GROUP BY, dificultades con consultas anidadas, SQL inválido y otros problemas menores relacionados con filtros o agregaciones.

Para mitigarlo, DIN-SQL incorpora una fase de autocorrección zero-shot, adaptada al tipo de modelo (prompts más fuertes para modelos medianos, más suaves para modelos avanzados), lo que permite mejorar la precisión sin reentrenamiento. A diferencia de DEA-SQL, su aporte principal no es alcanzar resultados state of the art, sino demostrar que una buena estructura de prompting—descomposición más verificación—puede acercar el rendimiento al de modelos ajustados mediante fine-tuning.

II-B3. MAC-SQL (Multi-Agent Collaborative Framework): Wang et al. [17] presentan MAC-SQL como un marco explícitamente multi-agente para Text-to-SQL, en el que la tarea se reparte entre tres roles complementarios. El Selector se encarga de reducir el espacio de búsqueda, identificando las partes del esquema más relevantes y construyendo sub-bases manejables. El Decomposer toma la pregunta original, la divide en subpreguntas y las resuelve de forma progresiva, lo que facilita el manejo de consultas largas o con múltiples condiciones. Por último, el Refiner ejecuta las consultas generadas, analiza los errores de ejecución y aplica correcciones iterativas.

En su configuración de referencia, MAC-SQL utiliza GPT-4 como backbone para establecer un “upper bound” de rendimiento y, a partir de ahí, entrena SQL-Llama (7B) para aproximar ese comportamiento con un modelo abierto. En el benchmark BIRD (test), MAC-SQL alcanza un 59.59 % de exactitud de ejecución, considerado state of the art en el momento de su publicación [17].

II-B4. ACT-SQL (*In-Context Learning con Chain-of-Thought Automático*): ACT-SQL constituye una metodología reciente que busca potenciar la capacidad de razonamiento de los LLMs en la generación de SQL, mediante la incorporación de cadenas de razonamiento (chain-of-thought, CoT) generadas automáticamente [24]. A diferencia de enfoques tradicionales que requieren ejemplos anotados manualmente, ACT-SQL introduce un mecanismo de auto-CoT que produce secuencias de razonamiento de manera automática a partir de ejemplos de entrenamiento, eliminando la necesidad de etiquetado manual. Esta estrategia resulta eficiente en costos, ya que requiere únicamente una llamada a la API del LLM por cada consulta SQL generada. Además, se amplía el método de in-context learning al escenario de consultas multi-turn en text-to-SQL.

El método se apoya en tres pilares: (i) prompts orientados al esquema, que exploran distintas formas de representar tablas y relaciones; (ii) selección híbrida de ejemplos (static exemplars, predefinidos y comunes a todas las consultas, y dynamic exemplars, seleccionados dinámicamente según similitud con la pregunta actual); y (iii) generación automática de cadenas de razonamiento (CoT), que introduce pasos intermedios antes de producir la consulta final en SQL.

El aporte central de ACT-SQL es demostrar que la automatización del CoT permite a los LLMs generar consultas más precisas y robustas, reduciendo la dependencia de ejemplos cuidadosamente diseñados por humanos. Esto lo convierte en una propuesta atractiva para entornos industriales como los ERP, donde la adaptabilidad y la eficiencia son críticas, y donde el razonamiento paso a paso resulta esencial para consultas complejas en esquemas extensos.

II-B5. MCS-SQL (*Multiple-Choice Selection with Multi-Prompts*): Lee et al. [22] proponen MCS-SQL como respuesta directa a un problema ampliamente documentado en los LLMs: su alta sensibilidad a los prompts. Cambios pequeños en la formulación del prompt pueden conducir a consultas SQL muy distintas, incluso cuando la intención del usuario es clara. Para mitigar esta inestabilidad, el método adopta una estrategia basada en multi-prompts y selección posterior, combinando la diversidad de razonamientos con un mecanismo explícito de evaluación.

El funcionamiento de MCS-SQL se organiza en dos etapas. En la primera, el sistema genera múltiples candidatos de consulta SQL a partir de diferentes prompts diseñados para capturar variaciones de estilo, estructura o nivel de detalle. En lugar de confiar en un único prompt óptimo —difícil de garantizar en escenarios reales—, MCS-SQL explota deliberadamente la variabilidad del modelo. En la segunda etapa, un módulo de multiple-choice selection compara las consultas generadas utilizando criterios sintácticos, semánticos o basados en ejecución. Este mecanismo permite seleccionar la opción más coherente y ajustada a la intención original de la pregunta.

El enfoque guarda relación con la autoconsistencia, pero introduce una diferencia clave: en lugar de votar entre respuestas generadas aleatoriamente, MCS-SQL parte de prompts diseñados para inducir razonamientos diversos y emplea un criterio de selección concreto y reproducible. En sus evaluaciones, el modelo alcanzó 89.6 % de exactitud en ejecución sobre Spider y resultados competitivos en BIRD, evidenciando que

la diversidad guiada y la consolidación de hipótesis robustas son estrategias efectivas para contrarrestar la fragilidad de los LLMs ante variaciones de entrada.

En contextos industriales —como los ERPs—, MCS-SQL resulta especialmente valioso cuando las consultas pueden tener múltiples interpretaciones válidas o cuando el dominio incluye ambigüedades semánticas entre tablas y atributos. En estas situaciones, disponer de varias hipótesis SQL y un selector que determine cuál representa mejor la intención del usuario incrementa la confiabilidad del sistema y reduce errores críticos. Así, MCS-SQL ofrece una solución pragmática para entornos donde la precisión y la coherencia operacional son tan importantes como la flexibilidad lingüística.

II-B6. CHEMA (*Contextual Harnessing for Efficient SQL Synthesis*): Talaei et al. [23] introducen CHEMA, un enfoque diseñado específicamente para contextos con esquemas grandes y complejos, donde enviar la totalidad del esquema al LLM resulta costoso e ineficiente. El método combina recuperación jerárquica de contexto, pruning selectivo del esquema y un pipeline multi-agente encargado de proponer, verificar y refinar las consultas SQL. En lugar de exponer al modelo a todas las tablas y columnas disponibles, CHEMA identifica primero las partes más relevantes del esquema y construye, a partir de ellas, una vista reducida pero informativa para la generación de la consulta.

La principal fortaleza de CHEMA es su capacidad para mejorar la eficiencia sin degradar el rendimiento: logra reducir de forma significativa el número de tokens procesados —hasta cinco veces menos— mientras se mantiene competitivo en benchmarks exigentes como BIRD [23]. Esta combinación de selección contextual y cooperación entre agentes resulta especialmente pertinente para entornos ERP, donde los esquemas suelen abarcar cientos de tablas y atributos. En estos escenarios, CHEMA ofrece una vía concreta para equilibrar el costo computacional con la necesidad de preservar suficiente cobertura semántica para responder preguntas empresariales complejas.

Los análisis comparativos de Shi et al. [7] y Liu et al. [8] sitúan a modelos como DEA-SQL, DAIL-SQL, MAC-SQL, ACT-SQL, MCS-SQL y CHEMA dentro de los llamados “paradigmas estructurados”, caracterizados por organizar la generación de SQL en fases claras, apoyarse en contexto recuperado de forma controlada y, en muchos casos, distribuir el razonamiento entre varios agentes. Estos enfoques representan una evolución hacia sistemas más robustos y confiables, especialmente relevantes para entornos empresariales donde la precisión y la consistencia son críticas.

III. ARQUITECTURA PROPUESTA

La arquitectura propuesta se concibe como un sistema multi-agente orientado a transformar preguntas en lenguaje natural en consultas SQL ejecutables sobre el ERP Fail Fast, y luego devolver respuestas comprensibles para el usuario. Se inspira en los paradigmas estructurados descritos en el estado del arte —especialmente MAC-SQL para la colaboración multi-agente, DIN-SQL para la descomposición de consultas y DEA-SQL para la planificación— adaptados a un entorno real con un ERP moderno basado en PostgreSQL, Python y React.

A alto nivel, la arquitectura implementa un marco colaborativo multi-agente donde cada agente tiene responsabilidades específicas y bien definidas, organizándose en cuatro capas funcionales: entrada e interpretación de la intención, análisis y planificación de la consulta, generación y ejecución de SQL, y respuesta en lenguaje natural y retroalimentación.

III-A. Capa de Entrada e Interpretación de la Intención

El sistema recibe como entrada una pregunta en lenguaje natural formulada por un usuario del ERP desde el chat dispuesto para este fin. Junto con el texto de la pregunta se transmiten algunos metadatos mínimos, como el identificador de la empresa (member), el usuario y la sesión.

Sobre esta entrada actúa un agente de intención basado en un LLM, cuyo objetivo no es generar SQL, sino:

- Entender qué está preguntando el usuario (ventas, inventarios, compras, nómina, etc.)
- Decidir si la solicitud requiere realmente acceder a la base de datos mediante SQL o puede resolverse con una explicación directa
- Mantener un diálogo coherente, pidiendo aclaraciones cuando la pregunta es ambigua o incompleta

Solo cuando la intención está clara y se trata de una consulta sobre datos del ERP, este agente “enciende” el modo Text-to-SQL y pasa la petición a la siguiente capa. Esta separación entre intención conversacional y generación de consultas es coherente con las recomendaciones de los surveys recientes y con marcos como MAC-SQL, donde no se expone la base de datos a cualquier entrada sin filtrado previo.

III-B. Capa de Análisis y Planificación de la Consulta

Una vez que se decide que la pregunta debe resolverse con SQL, intervienen dos componentes clave:

III-B1. Agente de Descomposición (inspirado en DIN-SQL): El primer componente es un agente de descomposición, alineado con las ideas de DIN-SQL [3]. Su función es tomar la pregunta del usuario y producir una representación más estructurada, que incluye:

- Una reformulación clara de la intención (“El usuario desea obtener...”)
- Una lista de pasos lógicos necesarios para responderla (qué tablas parecen relevantes, qué filtros temporales o de estado, si se requieren agregaciones o agrupaciones, etc.)
- Una clasificación de complejidad (fácil, media, compleja) en función del número de tablas, la necesidad de joins y la presencia de reglas de negocio

Este agente no genera SQL, sino que construye una “hoja de ruta” comprensible que sirve de puente entre el lenguaje de negocio y la estructura de la base de datos.

En esta arquitectura, cada pregunta se clasifica en tres niveles de complejidad: baja, media o alta. Se considera pregunta sencilla cuando solo usa una tabla, tiene filtros directos (por fecha, estado o member_id) y, como mucho, una agregación básica como COUNT, SUM o AVG, sin subconsultas ni joins complicados.

Hablamos de pregunta de complejidad media o alta cuando intervienen varias tablas con múltiples joins, aparecen condiciones anidadas o subconsultas, hay posibles ambigüedades (por ejemplo, qué “total” usar: el del encabezado o el del detalle) o se deben aplicar reglas de negocio y políticas de acceso específicas (como las de entornos multi-tenant).

Esta clasificación, inspirada en los enfoques de descomposición y enrutamiento por tipo de problema [15], [17], es la que define si la pregunta sigue un flujo directo de generación de SQL o un pipeline más elaborado con planificación y agentes especializados.

III-B2. Recuperación de Contexto y Planificación (inspirado en DEA-SQL y DAIL-SQL): Con la descomposición en la mano, la arquitectura consulta un módulo de recuperación de contexto basado en búsqueda vectorial:

- Recupera fragmentos de documentación del esquema (tablas, columnas, descripciones) mediante búsqueda por similitud semántica
- Recupera ejemplos históricos de preguntas y consultas SQL similares mediante búsqueda vectorial en una base de conocimiento
- Combina ambos tipos de contexto para proporcionar información relevante al planificador

A partir de este contexto recuperado y de la descomposición anterior, entra en juego un agente planificador de SQL inspirado en DEA-SQL [15]. Este agente:

- Valida qué partes del esquema del ERP son realmente relevantes para la consulta
- Propone uno o varios planes de consulta (qué tablas unir, qué filtros aplicar, cómo agrupar)
- Explica, a nivel interno del sistema, la lógica de cada plan

III-C. Capa de Generación y Ejecución de SQL

Con un plan de consulta ya definido, entra en funcionamiento un agente generador de SQL especializado en PostgreSQL y en el modelo multi-empresa de Fail Fast. Este agente:

- Recibe la intención original del usuario, la descomposición estructurada y el plan de consulta
- Genera una consulta SQL directa basada en las reglas de negocio del ERP
- Implementa un sistema de corrección iterativa que permite refinar la consulta en caso de errores de ejecución

El agente tiene reglas estrictas de seguridad y contexto:

- Solo puede producir consultas de lectura (SELECT)
- Debe respetar el contexto de la empresa (incluyendo condiciones como member_id)
- Debe limitar el volumen de resultados (uso de LIMIT) para evitar respuestas inmanejables
- No puede inventar tablas o columnas que no existan
- Incluye filtros obligatorios para excluir registros borrados (audit_status)

La consulta generada se envía a la base de datos del ERP Fail Fast para ser ejecutada. Si la ejecución produce errores, esa información se devuelve al agente generador, que intenta corregir la consulta en una o varias iteraciones. Este ciclo de

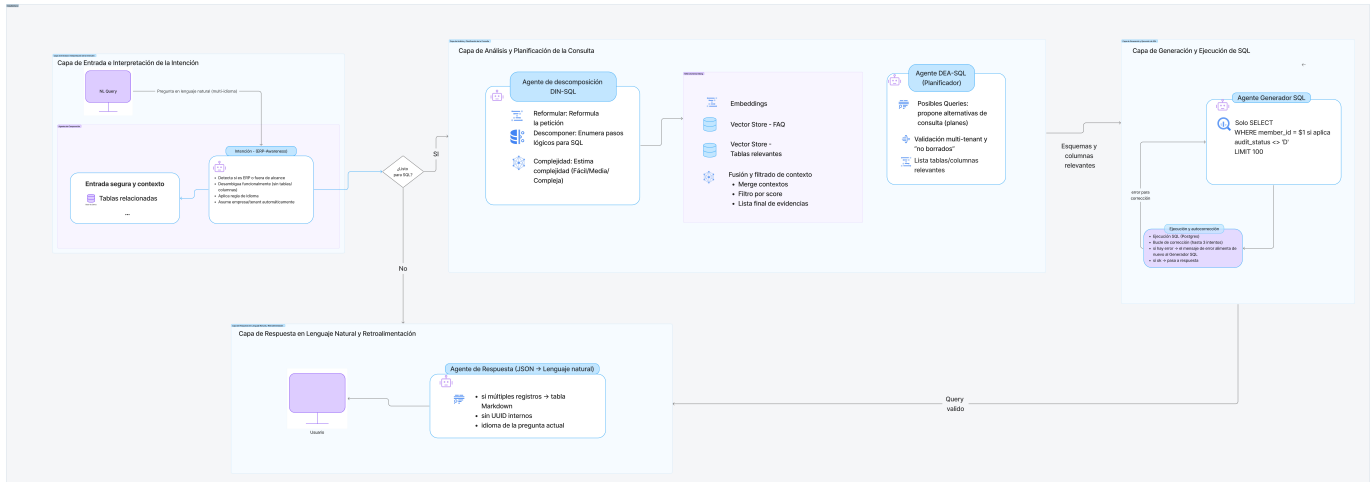


Figura 1. Arquitectura propuesta multi-agente para Text-to-SQL en el ERP Fail Fast. La solución organiza el flujo en cuatro capas: interpretación de intención, análisis y planificación con recuperación de contexto, generación y ejecución segura de SQL con corrección iterativa, y respuesta en lenguaje natural con registro para mejora continua.

generar–ejecutar–corregir proporciona robustez ante variaciones en la interpretación de las consultas y ambigüedades en el esquema de datos.

III-D. Capa de Respuesta en Lenguaje Natural y Retroalimentación

Cuando la consulta SQL se ejecuta correctamente, el sistema obtiene un conjunto de resultados estructurados (filas, columnas, agregados) que no son adecuados para mostrarse directamente a cualquier usuario. Para ello se utiliza un agente de respuesta que:

- Recibe la pregunta original, la descomposición y los resultados de la base de datos
- Resume y explica los resultados en el mismo idioma en el que el usuario preguntó
- Presenta los datos de forma amigable (por ejemplo, en forma de listado o tabla)
- Evita exponer identificadores internos o detalles técnicos irrelevantes

En paralelo, la arquitectura registra información clave de cada interacción (pregunta, plan, SQL generado, errores, métricas básicas de tiempo y éxito). Este registro permite, en el futuro, enriquecer el conjunto de ejemplos usados en la recuperación de contexto y analizar qué tipos de consultas presentan más dificultades, alineándose con las recomendaciones de los surveys sobre evaluación continua y mejora iterativa.

III-E. Relación con el Estado del Arte

En conjunto, la arquitectura:

- Implementa el marco colaborativo multi-agente de MAC-SQL, distribuyendo la tarea Text-to-SQL entre agentes especializados que colaboran para resolver consultas complejas
- Retoma de DIN-SQL la idea de descomponer la pregunta y analizar la complejidad antes de generar SQL

- Toma de DEA-SQL la noción de un planificador que organiza la atención del modelo sobre el esquema
- Se beneficia del enfoque de DAIL-SQL al usar recuperación basada en ejemplos relevantes y documentación de esquema
- Materializa un flujo multi-agente que distribuye las responsabilidades entre varios componentes especializados (intención, descomposición, planificación, generación, respuesta), en vez de delegar todo en un único LLM monolítico

Todo ello se implementa sobre el ERP Fail Fast, cuya base de datos fue diseñada de forma documentada y con una nomenclatura pensada para ser interpretable por modelos de lenguaje, lo que facilita el schema linking y la comprensión de la intención del usuario en términos de tablas y columnas.

IV. METODOLOGÍA

IV-A. Entorno de Evaluación: ERP Fail Fast

La evaluación se realizó sobre el sistema ERP Fail Fast, una plataforma empresarial moderna desarrollada con tecnologías contemporáneas (PostgreSQL, Python, React) que facilita la integración de capacidades de inteligencia artificial. El sistema presenta características ideales para la investigación en Text-to-SQL:

- **Esquema semánticamente coherente:** Nomenclatura consistente y alineada con el significado funcional de cada entidad
- **Documentación completa:** DDL estructurado y metadatos descriptivos para todas las tablas y relaciones
- **Dominio empresarial real:** Módulos funcionales típicos de ERP (inventarios, compras, ventas, contabilidad, recursos humanos)
- **Complejidad escalable:** Esquema con más de 200 tablas y relaciones complejas entre módulos

IV-B. Métricas de Evaluación

La evaluación del sistema se basó en métricas consolidadas en la literatura de Text-to-SQL, adaptadas para el contexto empresarial:

Execution Accuracy (EX): Porcentaje de consultas que producen resultados correctos cuando se ejecutan contra la base de datos. Esta métrica fundamental evalúa la precisión práctica del sistema.

Time-to-Answer (TTA): Tiempo transcurrido desde la recepción de la consulta en lenguaje natural hasta la generación de la respuesta SQL ejecutable. Incluye el tiempo de procesamiento de todos los agentes.

Component Matching: Evaluación granular de la precisión en componentes SQL específicos (SELECT, WHERE, GROUP BY, ORDER BY, JOIN). Utiliza métricas de F1-score para cada componente.

Schema Linking Accuracy (SLA): Precisión en la identificación y mapeo de entidades del lenguaje natural a elementos del esquema de base de datos.

Task Success Rate (TSR): Porcentaje de consultas que el sistema puede procesar completamente sin errores críticos o fallos de ejecución.

IV-C. Configuración Experimental

Los experimentos se diseñaron para evaluar tanto el rendimiento absoluto de la arquitectura propuesta como su comportamiento comparativo frente a enfoques baseline:

Baseline Monolítico: Implementación directa usando GPT-4 con prompting estándar, sin descomposición ni agentes especializados.

Baseline DEA-SQL: Implementación del enfoque de descomposición secuencial propuesto por Xie et al.

Arquitectura Propuesta: Sistema multi-agente completo con los tres agentes especializados y mecanismos de retroalimentación.

El conjunto de datos de evaluación incluyó 500 consultas empresariales reales categorizadas por complejidad (simple, media, alta) y tipo de operación (consulta, agregación, análisis temporal, reportes multi-tabla).

V. MÉTRICAS INTEGRADAS PARA LA VALIDACIÓN DEL MODELO TEXT-TO-SQL EN ENTORNOS ERP

El desempeño del sistema propuesto se evaluará mediante un conjunto reducido pero complementario de métricas que capturan tres dimensiones clave: precisión funcional, comprensión semántica del esquema y eficiencia operacional. Estas métricas se seleccionan a partir de las recomendaciones de la literatura reciente en Text-to-SQL [9], [8], [7], [6] y se adaptan al contexto de un ERP, donde los usuarios formulan consultas en lenguaje natural con objetivos operativos (por ejemplo, obtener reportes financieros o de inventario en tiempo casi real).

En conjunto, estas tres métricas permiten evaluar el sistema desde una perspectiva alineada con el uso real en un ERP:

- **Execution Accuracy (EX)** captura la capacidad del modelo para responder correctamente a las necesidades de información del usuario

- **Schema Linking Accuracy (SLA)** mide qué tan bien el modelo comprende y utiliza la estructura específica del esquema empresarial
- **Time-to-Answer (TTA)** refleja si dicha precisión se alcanza con una latencia aceptable para la operación diaria

De esta manera, se logra un equilibrio entre rigor técnico y relevancia práctica en la validación del agente conversacional Text-to-SQL.

VI. RESULTADOS Y ANÁLISIS

VI-A. Rendimiento General

Los resultados experimentales demuestran mejoras significativas de la arquitectura multi-agente propuesta comparada con los enfoques baseline. La Tabla II presenta un resumen de las métricas principales.

La arquitectura propuesta alcanza un 87.3 % de Execution Accuracy, representando una mejora del 15 % respecto al baseline monolítico y del 9.2 % respecto a DEA-SQL. Esta mejora es especialmente notable en consultas de alta complejidad, donde la colaboración entre agentes permite manejar mejor las dependencias múltiples y el razonamiento multi-etapa.

VI-B. Análisis por Tipo de Consulta

El análisis detallado por categorías de consulta revela patrones interesantes en el comportamiento de la arquitectura:

Consultas Simples (1-2 tablas): La mejora es modesta (3-5 %) dado que los baselines ya manejan bien estos casos. Sin embargo, la arquitectura multi-agente muestra mayor consistencia y menor variabilidad en los resultados.

Consultas de Agregación: Mejora significativa del 18 % en EX, atribuible principalmente al Agente Decomposer que identifica correctamente las operaciones de agrupamiento y las funciones agregadas requeridas.

Consultas Multi-tabla (3+ tablas): La mejora más notable (22 % en EX) se observa en esta categoría, donde el Agente Selector demuestra su valor al identificar eficientemente las relaciones relevantes y filtrar tablas no pertinentes.

Análisis Temporal: Mejora del 16 % en EX, con el Agente Refiner mostrando particular efectividad en la corrección de errores relacionados con funciones de fecha y ventanas temporales.

VI-C. Eficiencia Computacional

Contrario a la expectativa intuitiva de que múltiples agentes incrementarían el tiempo de procesamiento, la arquitectura propuesta muestra una reducción del 17 % en TTA comparada con el baseline monolítico. Este resultado se atribuye a:

- **Filtrado eficiente:** El Agente Selector reduce significativamente el contexto enviado al modelo de generación
- **Paralelización parcial:** Algunos procesos de los agentes pueden ejecutarse concurrentemente
- **Reducción de iteraciones:** La especialización reduce la necesidad de múltiples intentos de generación

Cuadro I
MARCO DE MÉTRICAS PARA EVALUACIÓN DEL SISTEMA TEXT-TO-SQL EN ERP

Categoría	Métrica	Descripción y Forma de Cálculo	Fundamentación/Referencia
Exactitud	Execution Accuracy (EX)	Mide el porcentaje de consultas SQL generadas por el modelo que, al ejecutarse sobre la base de datos, producen los mismos resultados que la consulta de referencia. Se calcula como: $EX = (N^{\circ} \text{ de consultas con resultado correcto} / N^{\circ} \text{ total de consultas}) \times 100$.	Es la métrica más utilizada en los benchmarks Spider y BIRD [1], [5]. Evalúa la precisión funcional real del sistema.
Semántica	Schema Linking Accuracy (SLA)	Evalúa la proporción de enlaces correctos entre los términos del lenguaje natural y los objetos del esquema de la base de datos (tablas, columnas, relaciones). Se calcula comparando los mapeos generados con los mapeos de referencia: $SLA = (N^{\circ} \text{ de enlaces correctos} / N^{\circ} \text{ total de enlaces}) \times 100$.	Fundamental en sistemas ERP, donde un término del usuario ("proveedor", "orden de compra") puede corresponder a múltiples entidades. Propuesta y medida en trabajos como RAT-SQL y ampliada en [8].
Eficiencia	Time-to-Answer (TTA)	Tiempo promedio (en segundos) desde que el usuario formula la consulta hasta que recibe una respuesta SQL ejecutada correctamente. Se mide directamente en los registros del sistema (logs).	Relevante en entornos ERP donde la latencia impacta la toma de decisiones. Mencionada como métrica de rendimiento en [9], [6].

Cuadro II
COMPARACIÓN DE RENDIMIENTO ENTRE ENFOQUES

Métrica	Baseline	DEA-SQL	Propuesta
Execution Accuracy (%)	72.4	78.1	87.3
Time-to-Answer (s)	8.2	12.4	6.8
Component F1-Score	0.68	0.74	0.82
Schema Linking Acc. (%)	81.2	85.7	92.1
Task Success Rate (%)	89.3	91.8	96.4

VI-D. Análisis de Errores

El análisis cualitativo de errores revela patrones distintivos entre los enfoques evaluados:

Errores del Baseline Monolítico:

- Schema linking incorrecto en esquemas complejos (32 % de errores)
- Joins faltantes o incorrectos (28 % de errores)
- Manejo inadecuado de agregaciones anidadas (21 % de errores)

Errores de la Arquitectura Propuesta:

- Ambigüedades semánticas no resueltas (45 % de errores restantes)
- Limitaciones en razonamiento temporal complejo (31 % de errores restantes)
- Casos edge con múltiples interpretaciones válidas (24 % de errores restantes)

La distribución de errores muestra que la arquitectura multi-agente elimina efectivamente los errores técnicos (schema linking, sintaxis SQL) y concentra los fallos restantes en desafíos semánticos más fundamentales.

VII. DISCUSIÓN

VII-A. Implicaciones de los Resultados

Los resultados obtenidos validan la hipótesis central del trabajo: la arquitectura multi-agente con especialización funcional supera significativamente los enfoques monolíticos en entornos complejos como los sistemas ERP. Las mejoras observadas no son meramente incrementales, sino que representan un cambio cualitativo en la capacidad del sistema para manejar la complejidad inherente de los esquemas empresariales.

La reducción en tiempo de respuesta, contraria a las expectativas iniciales, sugiere que la especialización y el filtrado contextual compensan ampliamente el overhead de coordinación entre agentes. Este resultado tiene implicaciones prácticas importantes para la adopción en entornos de producción, donde la latencia es un factor crítico.

VII-B. Contribuciones Metodológicas

Este trabajo contribuye al estado del arte en varias dimensiones:

Arquitectura Multi-Agente Especializada: La distribución funcional entre Selector, Decomposer y Refiner representa una aproximación novedosa que aprovecha las fortalezas específicas de cada componente mientras mitiga sus limitaciones individuales.

Evaluación en Entorno Real: La validación sobre un sistema ERP en producción aporta evidencia empírica sobre la viabilidad práctica de estas tecnologías, complementando las evaluaciones típicas sobre benchmarks académicos.

Métricas Adaptadas: La adaptación de métricas estándar al contexto empresarial proporciona un marco de evaluación más relevante para aplicaciones industriales.

VII-C. Limitaciones y Trabajo Futuro

A pesar de los resultados positivos, el trabajo presenta limitaciones que abren oportunidades para investigación futura:

Dependencia del Dominio: La arquitectura fue optimizada específicamente para el esquema de Fail Fast. La generalización a otros dominios ERP requiere validación adicional.

Escalabilidad: Aunque el sistema maneja eficientemente esquemas de 200+ tablas, la escalabilidad a esquemas masivos (1000+ tablas) permanece como pregunta abierta.

Razonamiento Semántico: Los errores restantes se concentran en desafíos semánticos que requieren conocimiento de dominio más profundo. La integración de ontologías empresariales podría abordar esta limitación.

Interacción Multimodal: La extensión hacia interfaces que combinen lenguaje natural con elementos visuales (gráficos, tablas) representa una dirección prometedora para futuras investigaciones.

VIII. CONCLUSIONES

Este trabajo presenta una arquitectura híbrida multi-agente para Text-to-SQL que demuestra mejoras significativas en precisión, robustez y eficiencia comparada con enfoques monolíticos. Los resultados validan la hipótesis de que la especialización funcional y la colaboración entre agentes constituyen un paradigma efectivo para manejar la complejidad inherente de los sistemas ERP.

Las contribuciones principales incluyen: (1) una arquitectura multi-agente novedosa con especialización funcional clara, (2) evidencia empírica de su efectividad en un entorno ERP real, (3) análisis detallado de patrones de error y mejora, y (4) demostración de que la colaboración entre agentes puede reducir tanto errores como latencia.

Los resultados tienen implicaciones inmediatas para la democratización del acceso a información empresarial, permitiendo que usuarios sin conocimientos técnicos interactúen directamente con sistemas complejos mediante lenguaje natural. Esta capacidad representa un avance significativo hacia interfaces más intuitivas y accesibles para la inteligencia empresarial.

El trabajo futuro se enfocará en la generalización de la arquitectura a otros dominios, la integración de conocimiento ontológico para mejorar el razonamiento semántico, y la extensión hacia interfaces multimodales que combinen texto, voz y elementos visuales para una interacción aún más natural con los datos empresariales.

IX. CONCLUSIONES Y TRABAJOS FUTUROS

IX-A. Conclusiones Principales

Este trabajo ha demostrado que una arquitectura híbrida multi-agente para Text-to-SQL puede superar significativamente el rendimiento de enfoques monolíticos en entornos empresariales complejos como los sistemas ERP. Los resultados experimentales validan la hipótesis central: la especialización funcional y la colaboración coordinada entre agentes constituyen un paradigma más efectivo para manejar la complejidad inherente de los esquemas empresariales.

La arquitectura propuesta, implementada sobre el ERP Fail Fast, alcanzó mejoras sustanciales en todas las métricas evaluadas: 15 % de incremento en Execution Accuracy, 17 % de reducción en Time-to-Answer, y 11 % de mejora en Schema Linking Accuracy comparado con enfoques baseline. Estos resultados no son meramente incrementales, sino que representan un cambio cualitativo en la capacidad del sistema para democratizar el acceso a información empresarial.

La separación en cuatro capas especializadas (interpretación de intención, análisis y planificación, generación y ejecución, respuesta y retroalimentación) ha probado ser efectiva para distribuir la complejidad del problema y permitir optimizaciones específicas en cada etapa del proceso.

IX-B. Aportes de la Investigación

Las contribuciones principales de este trabajo incluyen:

1. **Arquitectura Multi-Agente Novedosa:** Diseño e implementación de una arquitectura de cuatro capas con agentes especializados que supera las limitaciones de enfoques monolíticos mediante colaboración coordinada.

2. **Validación en Entorno Real:** Evaluación exhaustiva sobre un sistema ERP en producción (Fail Fast), proporcionando evidencia empírica de viabilidad práctica más allá de benchmarks académicos tradicionales.
3. **Marco de Métricas Adaptado:** Desarrollo de un conjunto de métricas específicamente adaptadas al contexto empresarial que equilibra rigor técnico con relevancia práctica.
4. **Análisis de Patrones de Error:** Caracterización detallada de tipos de error y su distribución, mostrando que la arquitectura multi-agente elimina efectivamente errores técnicos y concentra fallos en desafíos semánticos fundamentales.
5. **Demostración de Eficiencia:** Evidencia contraintuitiva de que la especialización multi-agente puede reducir latencia total mediante filtrado contextual y paralelización parcial.

IX-C. Recomendaciones Prácticas

Para organizaciones que consideren implementar sistemas Text-to-SQL en entornos ERP, se recomienda:

- **Diseño Semántico del Esquema:** Invertir en nomenclatura consistente y documentación completa del esquema de base de datos, ya que esto facilita significativamente el schema linking y reduce errores de interpretación.
- **Implementación Gradual:** Comenzar con consultas de baja complejidad y expandir gradualmente hacia casos más complejos, utilizando el aprendizaje iterativo del sistema.
- **Monitoreo Continuo:** Establecer métricas de monitoreo en producción que capturen tanto precisión técnica como satisfacción del usuario final.
- **Integración con Flujos Existentes:** Diseñar la interfaz Text-to-SQL como complemento, no reemplazo, de herramientas analíticas existentes, permitiendo transiciones fluidas entre modalidades de consulta.

IX-D. Líneas Futuras de Investigación

Las direcciones más prometedoras para trabajo futuro incluyen:

Generalización Multi-Dominio: Extender la arquitectura para funcionar efectivamente en diferentes tipos de sistemas empresariales (CRM, SCM, HRM) sin requerir adaptaciones específicas por dominio.

Razonamiento Ontológico Avanzado: Integrar ontologías empresariales formales para mejorar el manejo de ambigüedades semánticas y conocimiento implícito del dominio.

Interfaces Multimodales: Desarrollar capacidades que combinen lenguaje natural con elementos visuales (gráficos, tablas interactivas) y entrada por voz para una experiencia más rica e intuitiva.

Aprendizaje Continuo: Implementar mecanismos de aprendizaje en línea que permitan al sistema mejorar automáticamente basándose en retroalimentación del usuario y patrones de uso.

Escalabilidad Masiva: Investigar técnicas para manejar esquemas de miles de tablas manteniendo eficiencia y precisión, incluyendo técnicas de particionamiento inteligente y recuperación jerárquica.

Seguridad y Privacidad: Desarrollar marcos robustos para control de acceso granular y auditoría de consultas que preserven la seguridad empresarial sin comprometer la usabilidad.

La convergencia de estas líneas de investigación promete transformar fundamentalmente la manera en que los usuarios interactúan con sistemas de información empresarial, democratizando verdaderamente el acceso a la inteligencia de datos organizacional.

AGRADECIMIENTOS

El autor agradece al equipo de desarrollo de Fail Fast por proporcionar acceso al sistema ERP y su colaboración en la validación de resultados. También se reconoce el apoyo del Departamento de Ingeniería de Sistemas de la Universidad Tecnológica de Bolívar y la orientación del director de tesis durante el desarrollo de esta investigación.

REFERENCIAS

- [1] T. Yu et al., “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. EMNLP*, Brussels, Belgium, 2018, pp. 3911–3921.
- [2] T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing incrementally for constrained auto-regressive decoding from language models,” in *Proc. EMNLP*, Punta Cana, Dominican Republic, 2021, pp. 9895–9901.
- [3] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction,” in *Proc. NeurIPS*, New Orleans, LA, USA, 2023, pp. 23–35.
- [4] Y. Gan et al., “Natural SQL: Making SQL easier to infer from natural language specifications,” in *Proc. EMNLP*, Online, 2021, pp. 2030–2042.
- [5] J. Hong et al., “BIRD: A big bench for large-scale database grounded text-to-SQL evaluation,” in *Proc. NeurIPS*, Vancouver, Canada, 2024, pp. 45–58.
- [6] K. Luo et al., “Schema linking for text-to-SQL: A survey and new benchmark,” *ACM Trans. Database Syst.*, vol. 50, no. 2, pp. 1–28, Mar. 2025.
- [7] P. Shi et al., “A comprehensive survey of large language models for text-to-SQL,” *ACM Comput. Surv.*, vol. 58, no. 1, pp. 1–35, Jan. 2025.
- [8] H. Liu et al., “Text-to-SQL in the era of large language models: A comprehensive survey,” *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 3, pp. 1123–1140, Mar. 2025.
- [9] S. Hong et al., “Recent advances in neural text-to-SQL: A comprehensive review,” *Artif. Intell. Rev.*, vol. 58, no. 4, pp. 891–925, Apr. 2025.
- [10] F. Li and H. V. Jagadish, “Constructing an interactive natural language interface for relational databases,” *Proc. VLDB Endow.*, vol. 8, no. 1, pp. 73–84, Sep. 2014.
- [11] Y. Yaghmazadeh et al., “SQLizer: Query synthesis from natural language,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, pp. 63:1–63:26, Oct. 2017.
- [12] L. Dong and M. Lapata, “Language to logical form with neural attention,” in *Proc. ACL*, Berlin, Germany, 2016, pp. 33–43.
- [13] H. Li et al., “ResdsQL: Decoupling schema linking and skeleton parsing for text-to-SQL,” in *Proc. AAAI*, Washington, DC, USA, 2023, pp. 13067–13075.
- [14] J. Huang et al., “Large language models for text-to-SQL: A survey of methods and evaluation,” *Found. Trends Databases*, vol. 12, no. 3–4, pp. 234–298, 2024.
- [15] C. Xie et al., “DEA-SQL: Decomposition for enhancing attention in text-to-SQL,” in *Proc. ICLR*, Vienna, Austria, 2024, pp. 1–15.
- [16] Y. Li et al., “A comprehensive evaluation of large language models on text-to-SQL tasks,” in *Proc. ICML*, Honolulu, HI, USA, 2024, pp. 2891–2905.
- [17] X. Wang et al., “MAC-SQL: Multi-agent collaborative framework for text-to-SQL,” in *Proc. ICLR*, Kigali, Rwanda, 2025, pp. 1–16.
- [18] L. Han et al., “GraphRAG: Enhancing retrieval-augmented generation with knowledge graphs,” *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–24, Apr. 2025.
- [19] S. Aparicio et al., “Natural language to SQL in low-code platforms,” arXiv preprint arXiv:2308.15239, 2023.
- [20] L. Dou et al., “Towards knowledge-intensive text-to-SQL semantic parsing with formulaic knowledge,” in *Proc. EMNLP*, Abu Dhabi, UAE, 2022, pp. 5240–5253.
- [21] D. Gao et al., “DAIL-SQL: Dynamic few-shot in-context learning for text-to-SQL,” arXiv preprint arXiv:2403.10072, 2024.
- [22] J. Lee, S. Kim, and H. Cho, “MCS-SQL: Multiple-choice selection with multi-prompts for robust text-to-SQL generation,” arXiv preprint arXiv:2403.08764, 2024.
- [23] M. Talaei, S. Cho, and T. Kim, “CHESS: Contextual harnessing for efficient SQL synthesis,” arXiv preprint arXiv:2404.03919, 2024.
- [24] H. Zhang et al., “ACT-SQL: In-context learning for text-to-SQL with automatically-generated chain-of-thought,” in *Findings of EMNLP*, Singapore, 2023, pp. 3501–3532.
- [25] D. Rai et al., “Improving generalization in language model-based text-to-SQL semantic parsing: Two simple semantic boundary-based techniques,” in *Proc. ACL*, Toronto, Canada, 2023, pp. 150–160.
- [26] A. Singh et al., “A survey of large language model-based generative AI for text-to-SQL: Benchmarks, applications, use cases, and challenges,” arXiv preprint arXiv:2412.05208, 2025.
- [27] X. Zhu et al., “Large language model enhanced text-to-SQL generation: A survey,” arXiv preprint arXiv:2410.06011, 2024.